

Reliable Data Delivery: Error Detection and Correction



BENNETT
UNIVERSITY
TIMES OF INDIA GROUP

School of Computer Science Engineering and Technology
Bennett University
Greater Noida – 201310

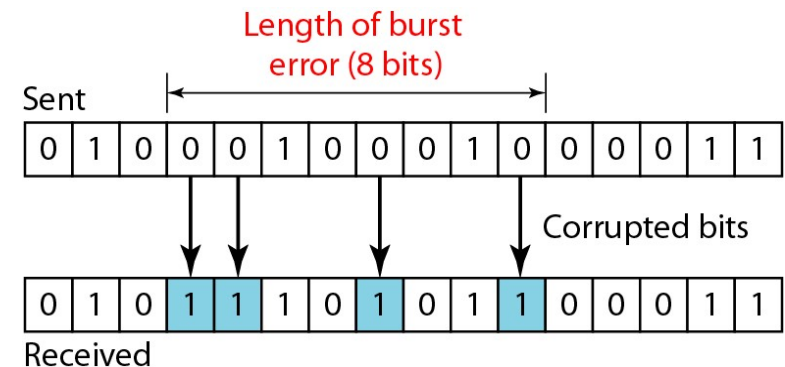
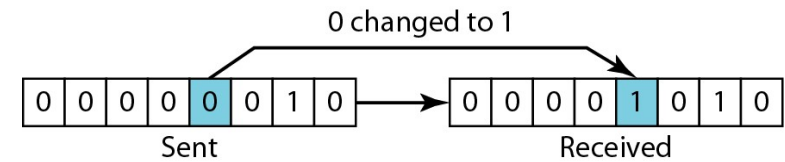
Outline

Types of Errors, Detection and Correction

Hamming Distance and CRC

Types of Error

- ❖ **Single Bit Error:** The term *single-bit error* means that only 1 bit of a given data unit (such as a byte, character, or packet) is changed from 1 to 0 or from 0 to 1.
- ❖ **Burst Error:** The term burst error means that 2 or more bits in the data unit have changed from 1 to 0 or from 0 to 1.
- ❖ Single-bit errors are the least likely type of error in serial data transmission. To understand why, imagine data sent at 1 Mbps. This means that each bit lasts only $1/1,000,000$ s. For a single-bit error to occur, the noise must have a duration of only 1 ns, which is very rare; noise normally lasts much longer than this. A burst error is more likely to occur than a single-bit error. The duration of noise is normally longer than the duration of 1 bit, which means that when noise affects data, it affects a set of bits.

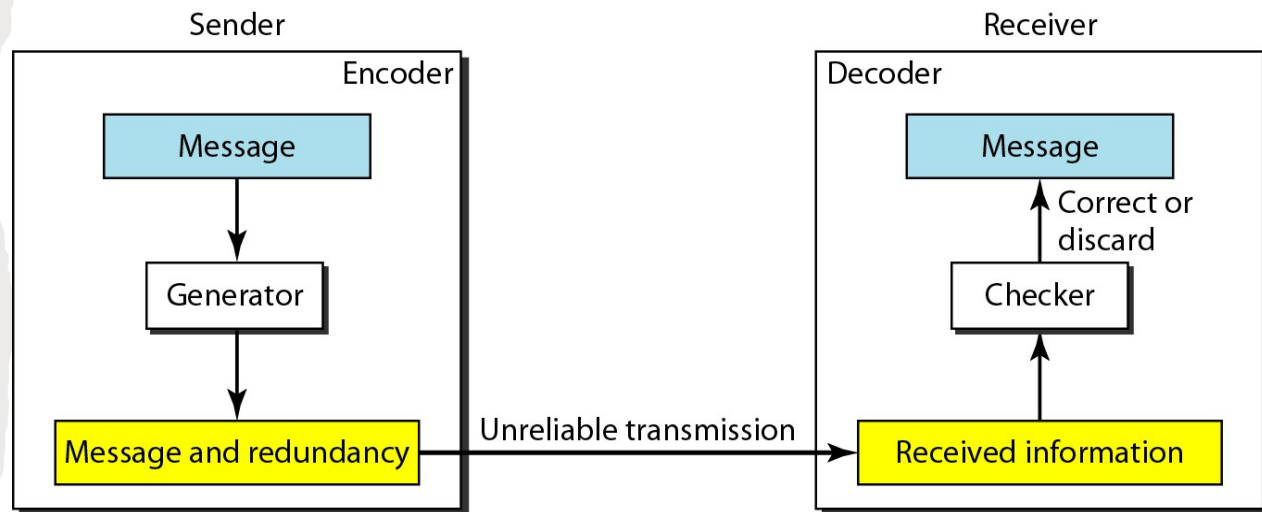


Redundancy

- ❖ To detect or correct errors, we need to send extra (redundant) bits with data.
- ❖ The receiver will be able to detect or correct the error using the extra information.
- ❖ Detection
 - Looking at the existence of any error, as YES or NO.
 - Retransmission if yes. (ARQ)
- ❖ Correction
 - Looking at both the number of errors and the location of the errors in a message.
 - Forward error correction. (FEC)

Coding

- ❖ Redundancy is achieved through various coding schemes. The sender adds redundant bits through a process that creates a relationship between the redundant bits and the actual data bits. The receiver checks the relationships between the two sets of bits to detect or correct the errors.
- ❖ The ratio of redundant bits to the data bits and the robustness of the process are important factors in any coding scheme.



Prerequisite: Modulo-2 Arithmetic

- ❖ Possible numbers are {0, 1}
- ❖ Arithmetic
 - Addition
 - $0+0=0$, $0+1=1$, $1+0=1$, $1+1=2=0$
 - Subtraction
 - $0-0=0$, $0-1=-1=1$, $1-0=1$, $1-1=0$
 - Surprisingly, the addition and subtraction give the same result.
 - XOR (exclusively OR) can replace both addition and subtraction.

$$0 \oplus 0 = 0$$

$$1 \oplus 1 = 0$$

a. Two bits are the same, the result is 0.

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

b. Two bits are different, the result is 1.

	1	0	1	1	0
$\oplus$	1	1	1	0	0
	0	1	0	1	0

c. Result of XORing two patterns

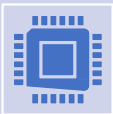
Block Coding



In block coding, we divide our message into blocks, each of k bits, called datawords. We add r redundant bits to each block to make the length $n = k + r$. The resulting n -bit blocks are called codewords.



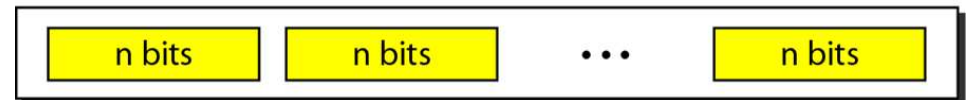
With k bits, we can create a combination of 2^k datawords; with n bits, we can create a combination of 2^n codewords. Since $n > k$, the number of possible codewords is larger than the number of possible datawords.



The block coding process is one-to-one; the same dataword is always encoded as the same codeword. This means that we have $2^n - 2^k$ codewords that are not used.



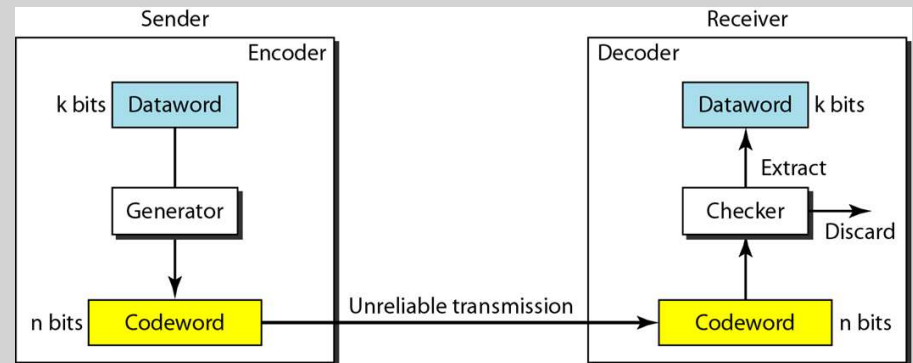
2^k Datawords, each of k bits



2^n Codewords, each of n bits (only 2^k of them are valid)

Error Detection

- ❖ A receiver can detect a change if the original codeword if
 - The receiver has a list of valid codewords, and
 - The original codeword has changed to an invalid one.



Error Detection Example

- Let us assume that $k = 2$ and $n = 3$ and assume the following table.
 - ❖ Assume the sender encodes the dataword 01 as 011 and sends it to the receiver. Consider the following cases:
 - The receiver receives 011. It is a valid codeword. The receiver extracts the dataword 01 from it.
 - The codeword is corrupted during transmission, and 111 is received. This is not a valid codeword and is discarded.
 - The codeword is corrupted during transmission, and 000 is received. This is a valid codeword. The receiver incorrectly extracts the dataword 00. Two corrupted bits have made the error undetectable.

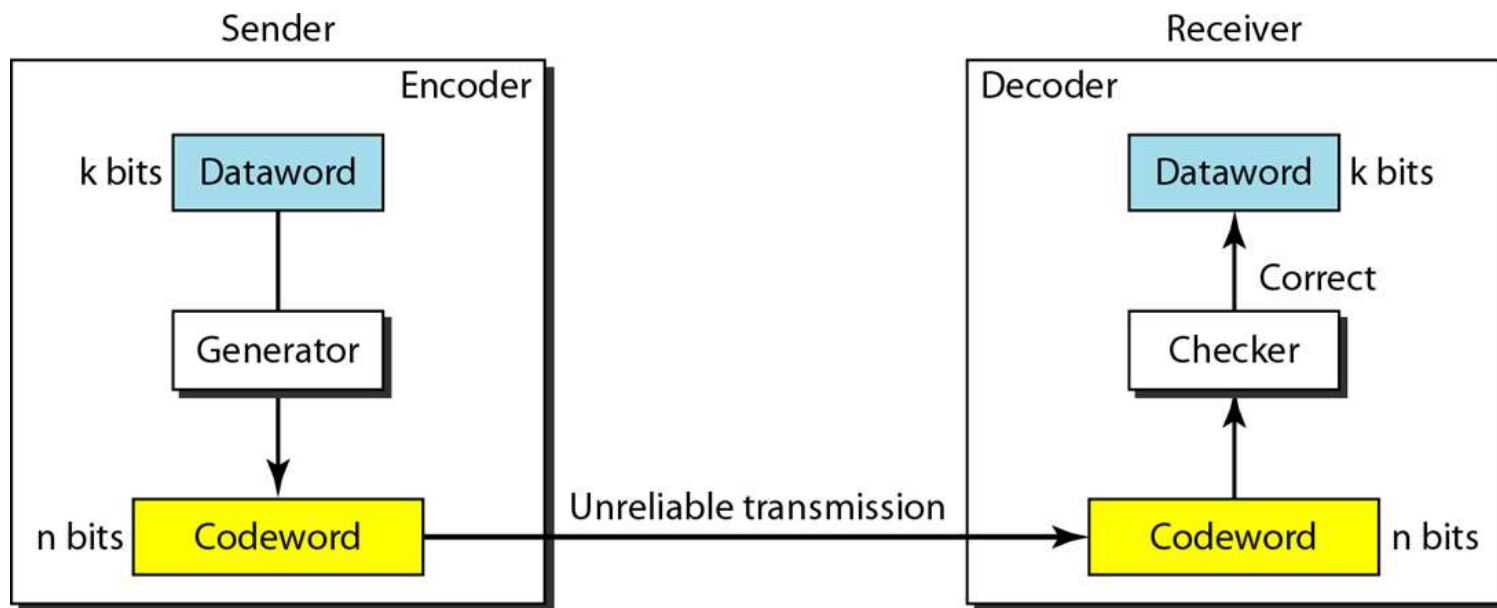
<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

Note

- An error-detecting code can detect only the types of errors for which it is designed; other types of errors may remain undetected.
- ❖ The previous example
 - Is designed for detecting 1-bit error,
 - Cannot detect 2-bit error, and
 - Cannot find the location of the 1-bit error.

Error Correction

- ❖ The receiver needs to find (or guess) the original codeword sent.
- ❖ Need more redundancy than for error detection.



Error Correction Example

<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

❖ Add 3 redundant bits to the 2-bit dataword to make 5-bit codewords as follows:

Assume the dataword is 01.

- ❑ The sender creates the codeword 01011.
- ❑ The codeword is corrupted during transmission, and 01001 is received. The receiver finds that the received codeword is not in the table.
- Assuming that there is only 1 bit corrupted, uses the following strategy to guess the correct dataword.
- ✓ Comparing the received codeword with the first codeword in the table (01001 versus 00000), the receiver decides that the first codeword is not the one that was sent because there are two different bits.
- ✓ By the same reasoning, the original codeword cannot be the third or fourth one in the table.
- ✓ The original codeword must be the second one in the table because this is the only one that differs from the received codeword by 1 bit. The receiver replaces 01001 with 01011 and consults the table to find the dataword 01.

Hamming Distance

❖ The Hamming distance between two words (of the same size) is the number of differences between the corresponding bits. We show the Hamming distance between two words x and y as $d(x, y)$.

✓ The Hamming distance $d(000, 011)$ is 2 because $000 \text{ XOR } 011 = 011$ (two 1's)

✓ The Hamming distance $d(10101, 11110)$ is 3 because $10101 \text{ XOR } 11110 = 01011$ (three 1's)

➤ The minimum Hamming distance is the smallest Hamming distance between all possible pairs in a set of words.

Minimum Hamming Distance Example

❖ Given the Hamming distances.

$d(000,011)=2$, $d(000,101)=2$, $d(000,110)=2$
 $d(011,101)=2$, $d(011,110)=2$, $d(101,110)=2$

The $d_{\min}$ in this case is 2.

❖ Given the Hamming distances

We first find all the Hamming distances.

$d(0000,01011)=3$, $d(00000,10101)=3$, $d(00000,11110)=4$,
 $d(01011,10101)=4$, $d(01011,11110)=3$, $d(10101,11110)=3$

The $d_{\min}$ in this case is 3.

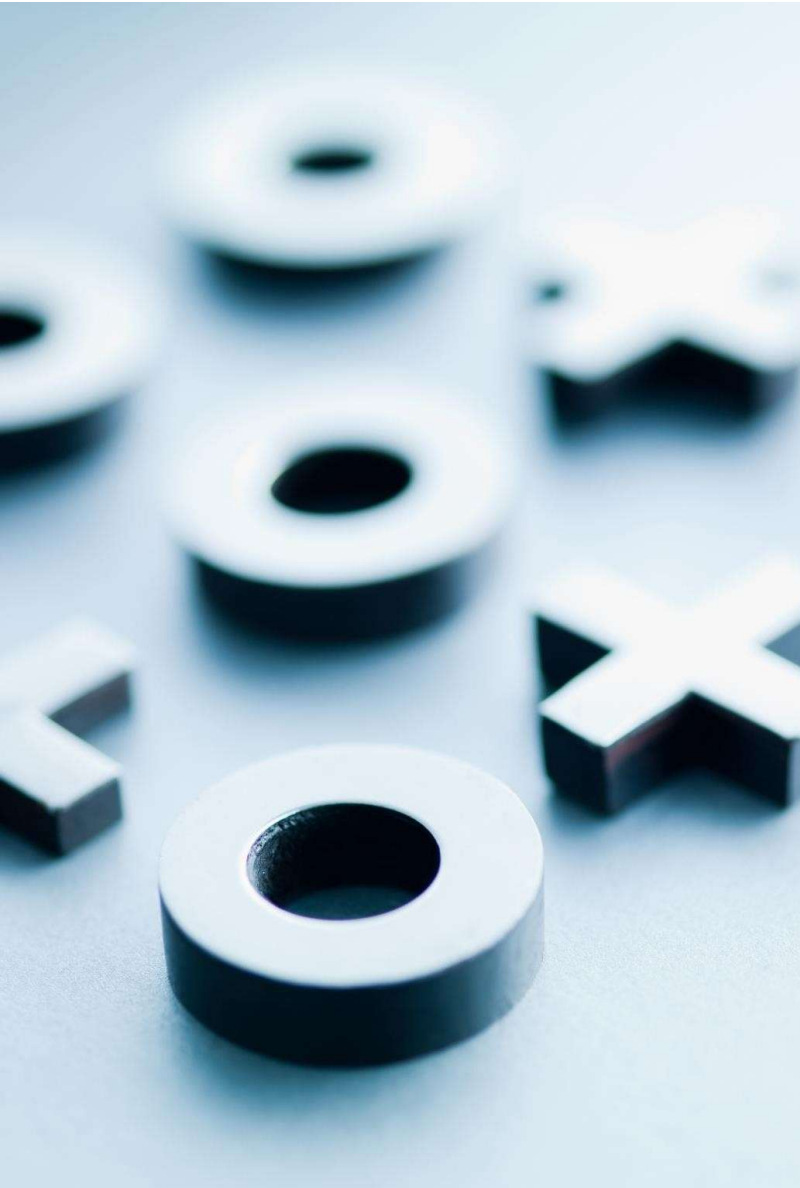
Any coding scheme needs to have at least three parameters: the codeword size n , the dataword size k , and the minimum Hamming distance $d_{\min}$.

A coding scheme C is written as $C(n, k)$ with a separate expression for $d_{\min}$.

For example, we can call our first coding scheme $C(3, 2)$ with $d_{\min} = 2$ and our second coding scheme $C(5, 2)$ with $d_{\min} = 3$.



The Three Parameters

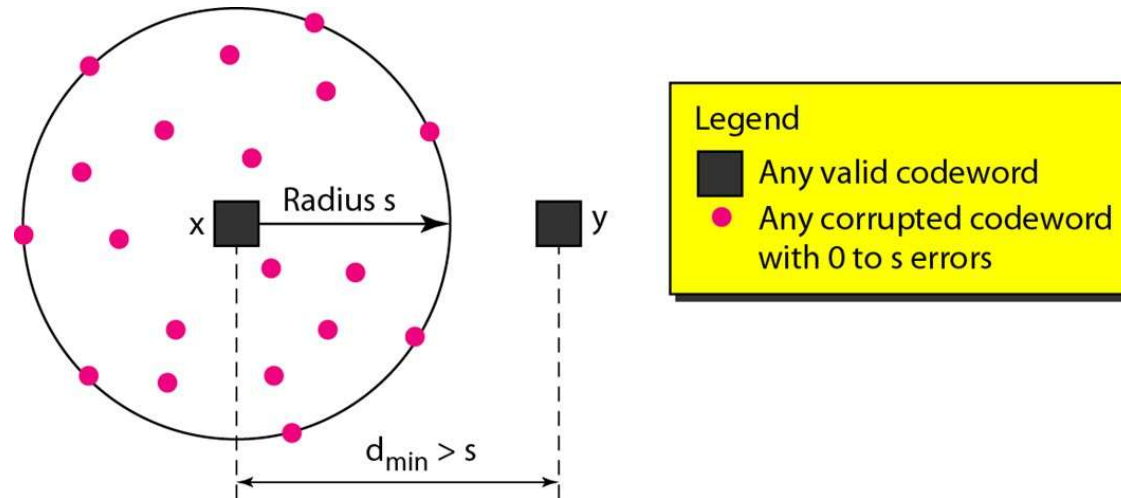


Hamming Distance and Error Detection

- When a codeword is corrupted during transmission, the Hamming distance between the sent and received codewords is the number of bits affected by the error. In other words, the Hamming distance between the received codeword and the sent codeword is the number of bits that are corrupted during transmission.
- For example, if the codeword 00000 is sent and 01101 is received, 3 bits are in error and the Hamming distance between the two is $d(00000, 01101) = 3$.

Minimum Distance for Error Detection

If s errors occur during transmission, the Hamming distance between the sent codeword and received codeword is s . If our code is to detect up to s errors, the minimum distance between the valid codes must be $s + 1$, so that the received codeword does not match a valid codeword. In other words, if the minimum distance between all valid codewords is $s + 1$, the received codeword cannot be erroneously mistaken for another codeword.



Example: Minimum Distance for Error Detection

<i>Datawords</i>	<i>Codewords</i>
00	000
01	011
10	101
11	110

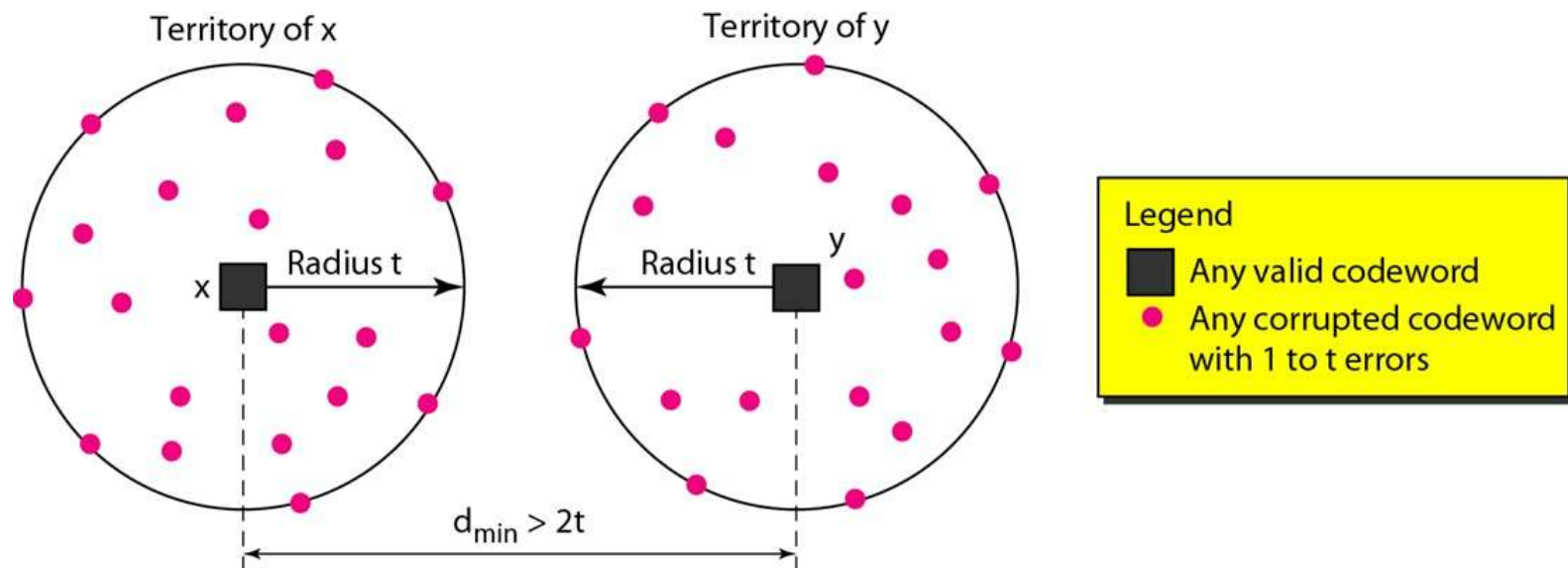
This code guarantees detection of only a single error. For example, if the third codeword (10 1) is sent and one error occurs, the received codeword does not match any valid codeword. If two errors occur, however, the received codeword may match a valid codeword and the errors are not detected.

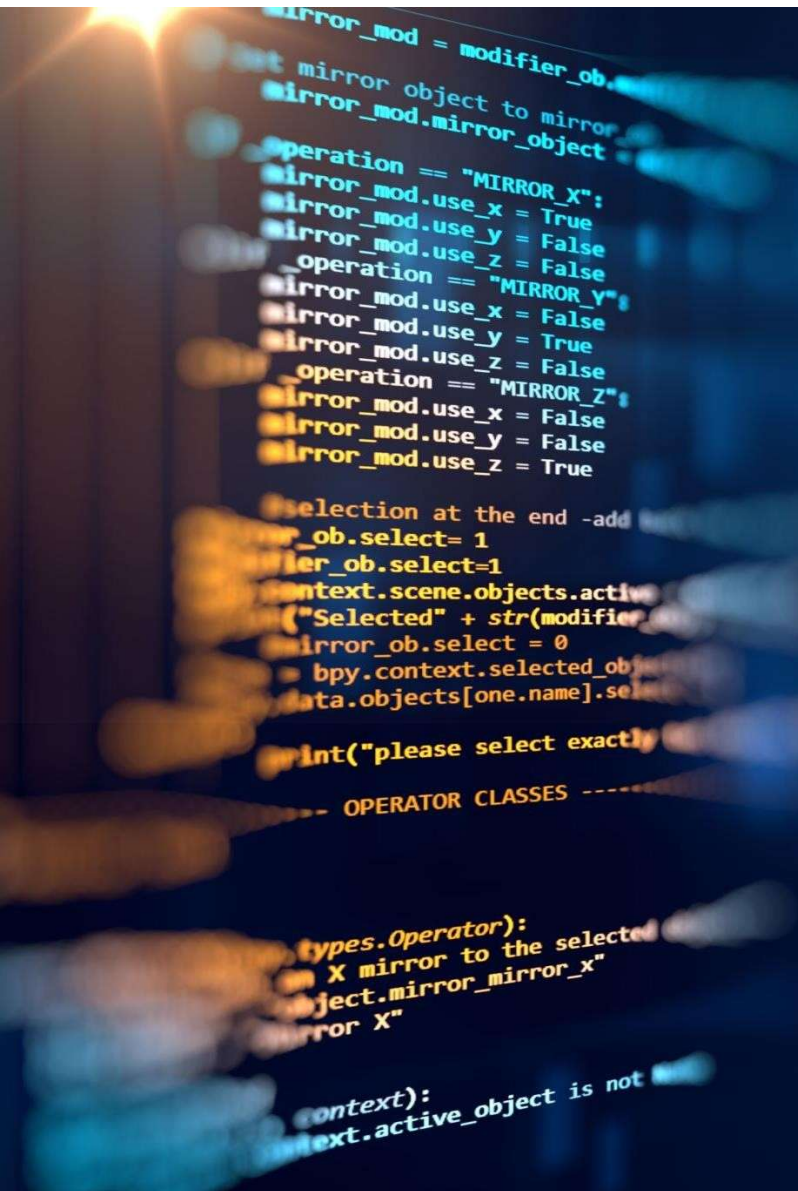
<i>Dataword</i>	<i>Codeword</i>
00	00000
01	01011
10	10101
11	11110

This code can detect up to two errors. Again, we see that when any of the valid codewords is sent, two errors create a codeword which is not in the table of valid codewords. The receiver cannot be fooled. However, some combinations of three errors change a valid codeword to another valid codeword. The receiver accepts the received codeword and the errors are undetected.

Minimum Distance and Correction

- The decision is based on the concept of territory, an exclusive area surrounding the codeword. Each valid codeword has its own territory.
- We use a geometric approach to define each territory. We assume that each valid codeword has a circular territory with a radius of t and that the valid codeword is at the center.
- To guarantee correction of up to t errors in all cases, the minimum Hamming distance in a block code must be $d_{\min} = 2t + 1$.





Example

A code scheme has a Hamming distance $d_{\min} = 4$. What is the error detection and correction capability of this scheme?

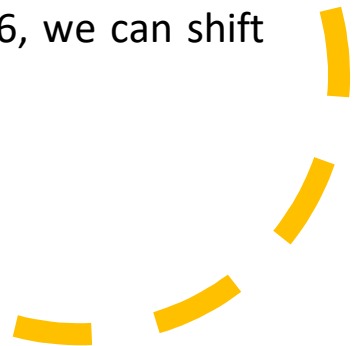
✓ Solution

- ☐ This code guarantees the detection of up to three errors ($s = 3$), but it can correct up to one error.
- ☐ In other words, if this code is used for error correction, part of its capability is wasted. Error correction codes need to have an odd minimum distance (3, 5, 7, ...).

$$b_1 = a_0 \quad b_2 = a_1 \quad b_3 = a_2 \quad b_4 = a_3 \quad b_5 = a_4 \quad b_6 = a_5 \quad b_0 = a_6$$

Cyclic Codes

- ❖ Cyclic codes are special linear block codes with one extra property. In a cyclic code, if a codeword is cyclically shifted (rotated), the result is another codeword. For example, if 1011000 is a codeword and we cyclically left-shift, then 0110001 is also a codeword. In this case, if we call the bits in the first word a_0 to a_6 and the bits in the second word b_0 to b_6 , we can shift the bits by using the following:

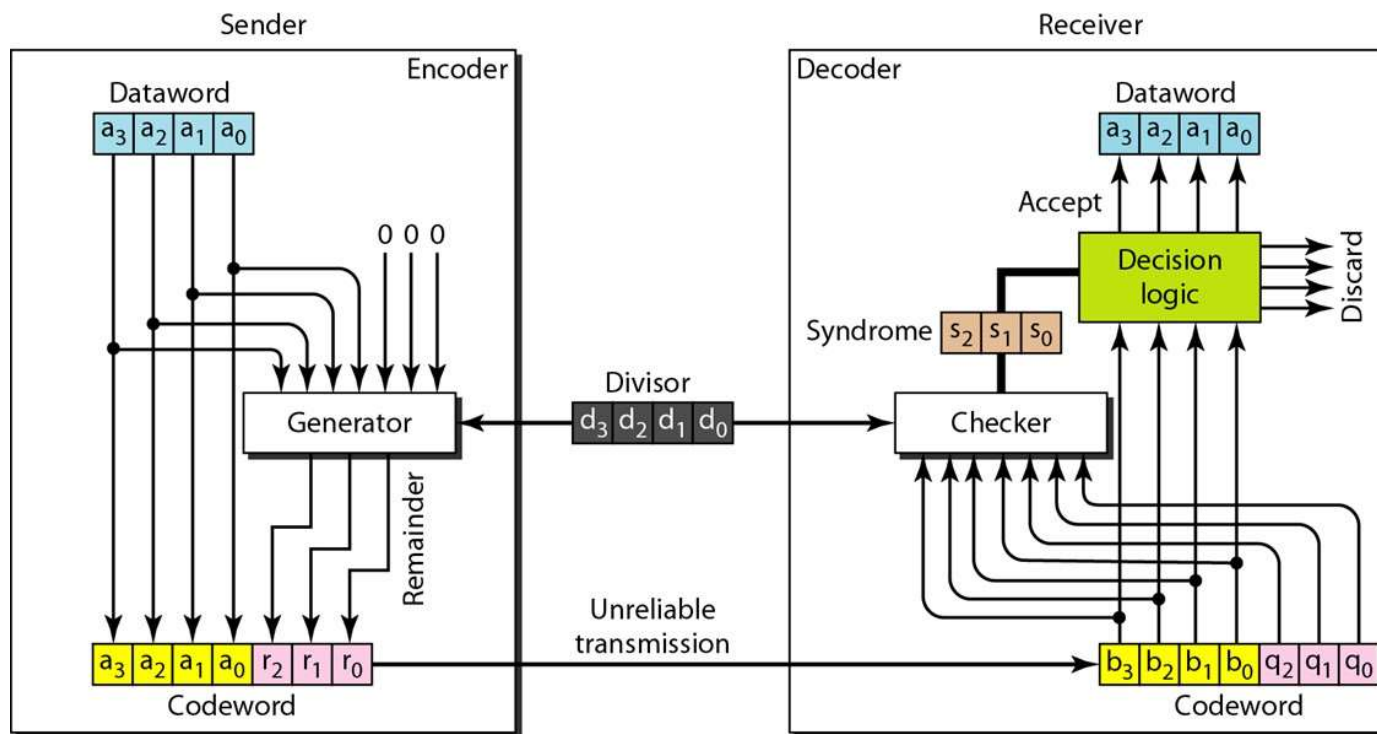


Cyclic Redundancy Check

- ❖ Cyclic codes called the cyclic redundancy check (CRC) that is used in networks such as LANs and WANs (Ethernet).
- ❖ In the encoder, the dataword has k bits (4 here); the codeword has n bits (7 here). The size of the dataword is augmented by adding $n - k$ (3 here) 0s to the right-hand side of the word. The n -bit result is fed into the generator. The generator uses a divisor of size $n - k + 1$ (4 here), predefined and agreed upon. The generator divides the augmented dataword by the divisor (modulo-2 division). The quotient of the division is discarded; the remainder is appended to the dataword to create the codeword.
- ❖ The decoder receives the possibly corrupted codeword. A copy of all n bits is fed to the checker which is a replica of the generator. The remainder produced by the checker is a syndrome of $n - k$ (3 here) bits, which is fed to the decision logic analyzer. The analyzer has a simple function. If the syndrome bits are all 0s, the 4 leftmost bits of the codeword are accepted as the dataword (interpreted as no error); otherwise, the 4 bits are discarded (error).

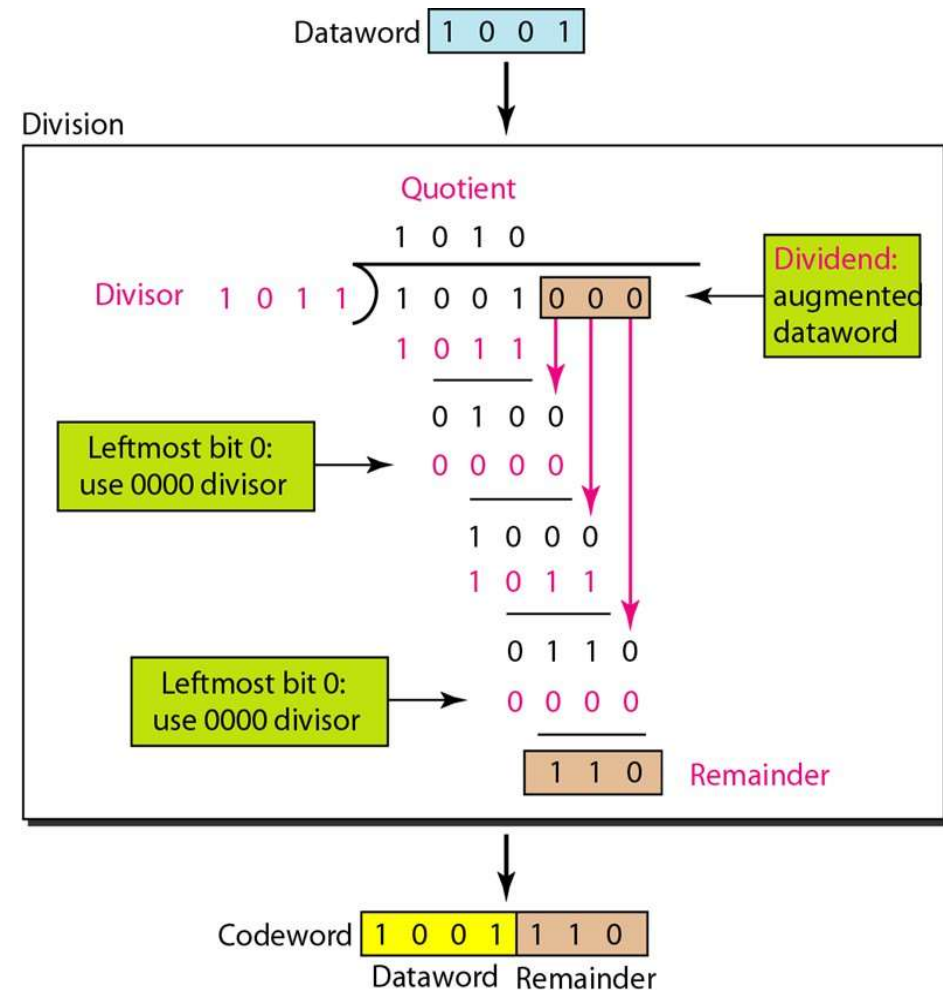


Architecture of CRC



Division in CRC Encoder

- The encoder takes the dataword and augments it with $n - k$ number of as. It then divides the augmented dataword by the divisor.
- The process of modulo-2 binary division is the same as the familiar division process we use for decimal numbers. However, in this case addition and subtraction are the same. We use the XOR operation to do both.

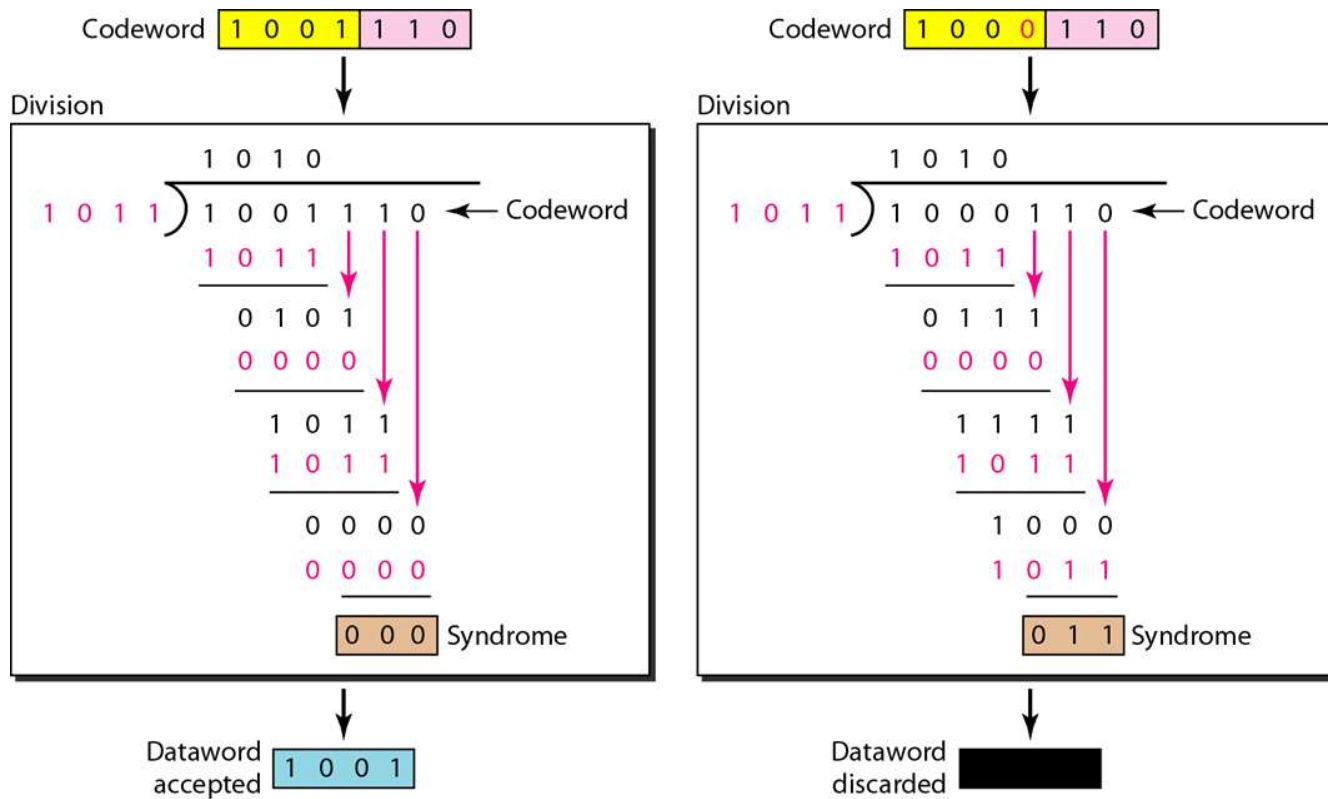


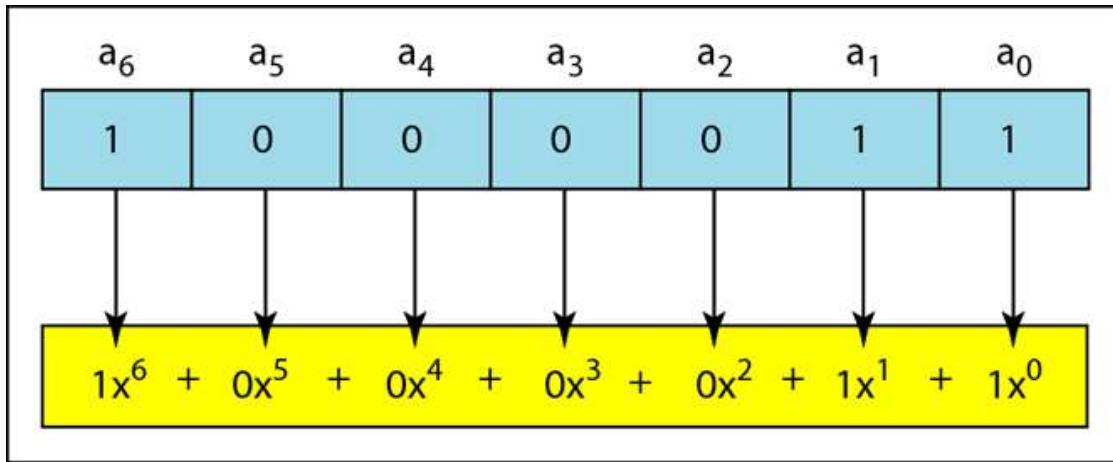
CRC Decoder

- The decoder does the same division as the encoder.
- The remainder of the division is the syndrome.
- ❑ If there is no error during communication, the syndrome is zero. The dataword is separated from the received codeword and accepted.
- ❑ If the syndrome is non-zero, then errors occur during communication.

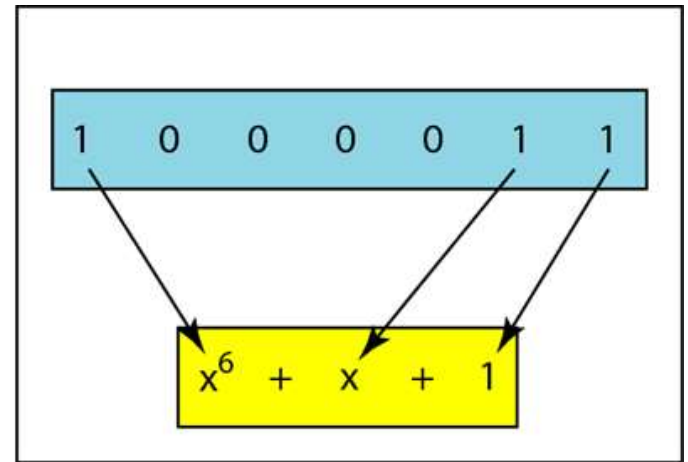


Division in the CRC decoder for two cases





a. Binary pattern and polynomial



b. Short form

Polynomials

- The binary vector can be represented by a polynomial.
 - Coefficients are either 0 or 1.
 - Power of each term represents the position of the bit.

$$\begin{array}{r}
 1011 \overline{) 1110} \\
 \underline{1011} \\
 111000 \\
 \underline{1011} \\
 10100 \\
 \underline{1011} \\
 10
 \end{array}$$

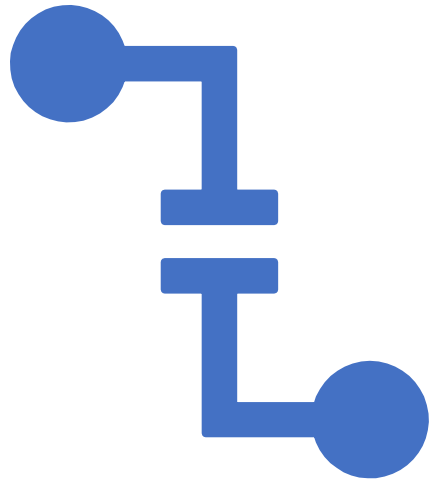
	Polynomial	Binary Vector
Generator, $g(x)$	x^3+x+1	1011
Data, $f(x)$	x^3+x^2	1100
$x^n f(x)$	x^6+x^5	1100000
Division $x^n f(x) \div g(x)$		
Codeword, $x^n f(x) - r(x)$	x^6+x^5+x	1100010

$$\begin{array}{r}
 x^3+x+1 \overline{) x^3+x^2+x} \\
 \underline{x^6+x^5} \\
 x^6+x^4+x^3 \\
 \underline{x^5+x^4+x^3} \\
 x^5+x^3+x^2 \\
 \underline{x^4+x^2} \\
 x^4+x^2+x \\
 \underline{} \\
 x
 \end{array}$$

Equivalence of Polynomial and Binary



- The divisor in a cyclic code is normally called the generator polynomial or simply the generator.
- In a cyclic code, the remainder of $(x^n f(x) - r(x) + e(x)) \% g(x)$
 - If $s(x) \neq 0$, one or more bits is corrupted.
 - If $s(x) = 0$, either
 - ✓ No bit is corrupted, or
 - ✓ Some bits are corrupted, but the decoder failed to detect them.
- In a cyclic code, those $e(x)$ errors that are divisible by $g(x)$ are not caught.



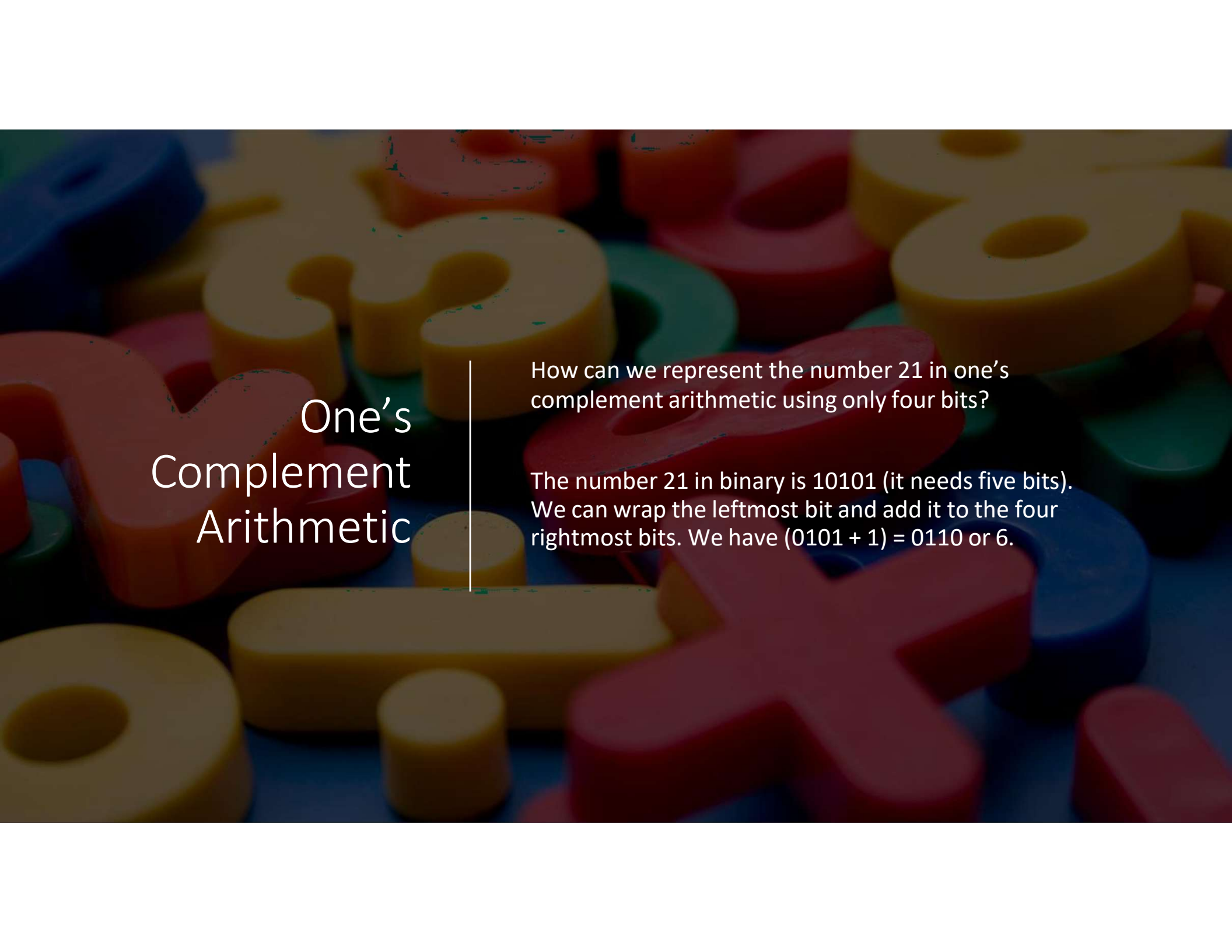
Capability of CRC

- If the generator has more than one term and the coefficient of x^0 is 1, all single errors can be caught.
- If a generator cannot divide x^{t+1} (t between 0 and $n - 1$), then all isolated double errors can be detected.
- A generator that contains a factor of $x+1$ can detect all odd-numbered errors.



Checksum

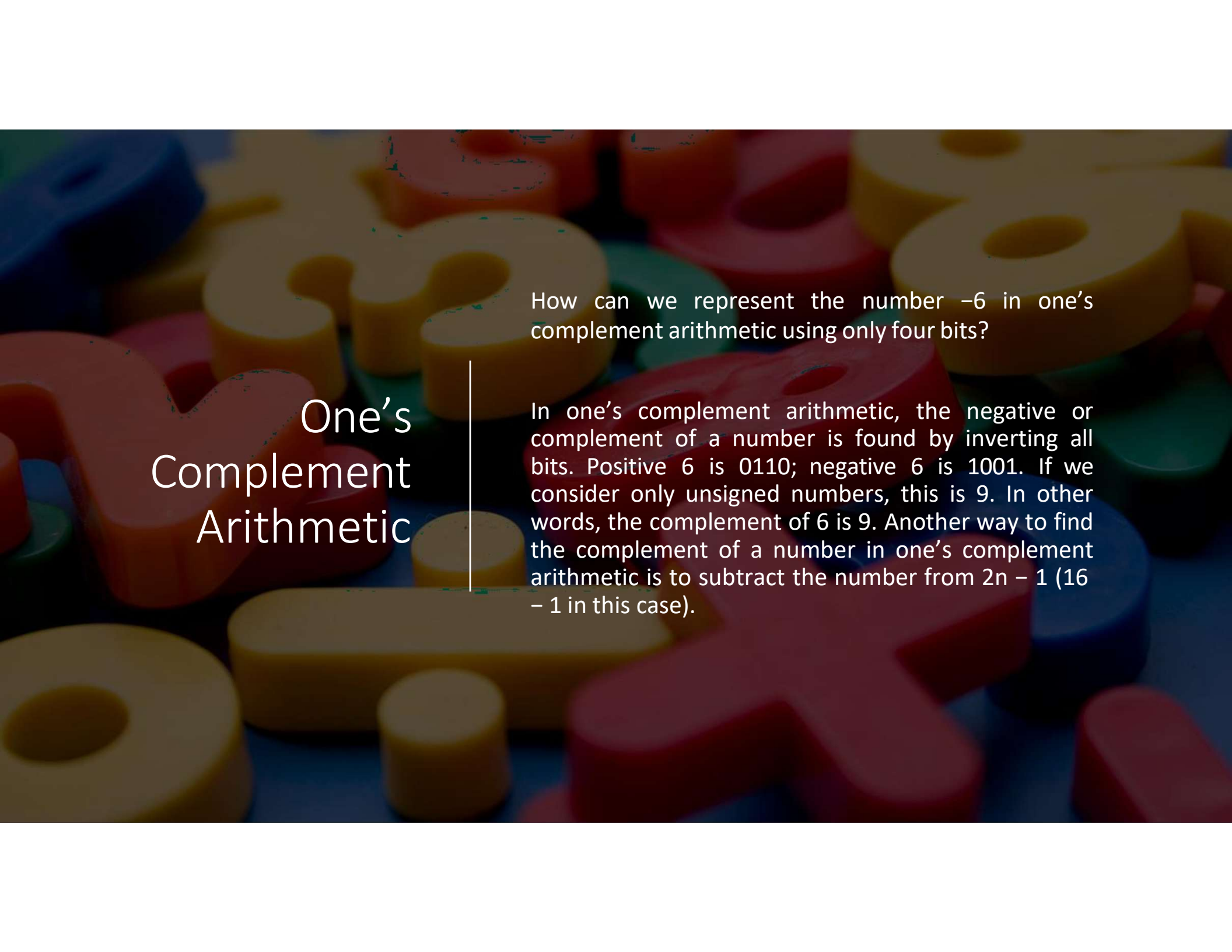
- ❖ The checksum is used on the Internet by several protocols although not at the data link layer.
- ❖ Suppose our data is a list of five 4-bit numbers that we want to send to a destination. In addition to sending these numbers, we send the sum of the numbers. For example, if the set of numbers is (7, 11, 12, 0, 6), we send (7, 11, 12, 0, 6, 36), where 36 is the sum of the original numbers. The receiver adds the five numbers and compares the result with the sum. If the two are the same, the receiver assumes no error, accepts the five numbers, and discards the sum. Otherwise, there is an error somewhere and the data are not accepted.
- ❖ We can make the job of the receiver easier if we send the negative (complement) of the sum, called the checksum. In this case, we send (7, 11, 12, 0, 6, -36). The receiver can add all the numbers received (including the checksum). If the result is 0, it assumes no error; otherwise, there is an error.



One's Complement Arithmetic

How can we represent the number 21 in one's complement arithmetic using only four bits?

The number 21 in binary is 10101 (it needs five bits). We can wrap the leftmost bit and add it to the four rightmost bits. We have $(0101 + 1) = 0110$ or 6.

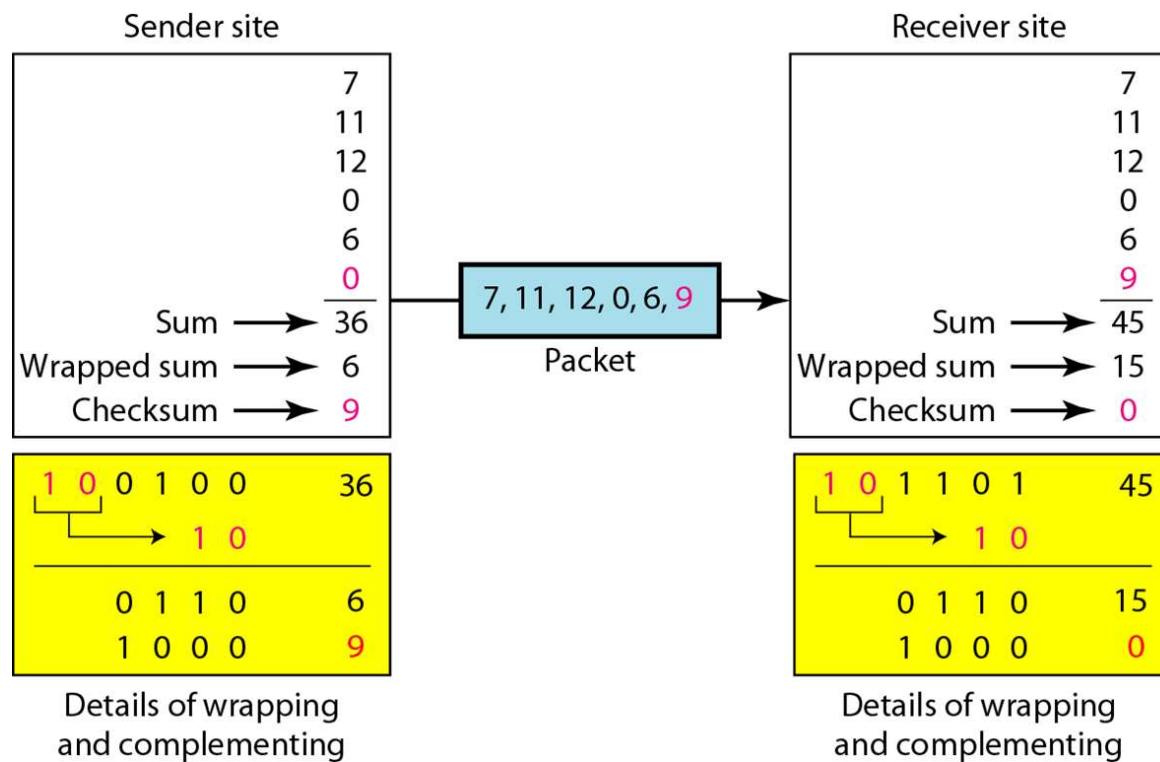


One's Complement Arithmetic

How can we represent the number -6 in one's complement arithmetic using only four bits?

In one's complement arithmetic, the negative or complement of a number is found by inverting all bits. Positive 6 is 0110; negative 6 is 1001. If we consider only unsigned numbers, this is 9. In other words, the complement of 6 is 9. Another way to find the complement of a number in one's complement arithmetic is to subtract the number from $2^n - 1$ ($16 - 1$ in this case).

Let's Redo the Example from Slide 35





Thank You